Activity Selection Problem

Design and Analysis of Algorithms (DAA)

Brief Explanation

The Activity Selection Problem is a classic Greedy Algorithm problem. Given a set of activities, each with a start time and finish time, the goal is to select the maximum number of activities that do not overlap.

Greedy strategy: Sort the activities by increasing finish time, select the activity that finishes earliest, and then select each next activity whose start time is greater than or equal to the finish time of the last selected activity.

Code Screenshot

```cpp
int main() {
    int n;
    cout << "Enter number of activities: ";
    cin >> n;

    int start[100], finish[100];

    for (int i = 0; i < n; i++) {
        cout << "Enter start and finish time of activity " << i + 1 << ": ";
        cin >> start[i] >> finish[i];
    }

    // Sort activities by finish time
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (finish[j] > finish[j + 1]) {
                swap(finish[j], finish[j + 1]);
                swap(start[j], start[j + 1]);
            }
        }
    }

    cout << "\nSelected activities:\n";

    int lastFinish = -1;

    for (int i = 0; i < n; i++) {
        if (start[i] >= lastFinish) {
            cout << "(" << start[i] << ", " << finish[i] << ")\n";
            lastFinish = finish[i];
        }
    }

    return 0;
}
```

Algorithm Steps

1. Read the number of activities and their start and finish times.
2. Sort all activities by finish time using bubble sort.
3. Select the first activity in the sorted list.
4. For every remaining activity, select it if its start time is at least the last selected finish time.
5. Print the selected activities.

Time and Space Complexity

Time Complexity: O(n²) — the bubble sort uses two nested loops. The selection scan takes O(n), so sorting dominates the overall running time.

Auxiliary Space Complexity: O(1) — the program uses fixed-size arrays and a few variables, and sorts the activities in place. (The arrays store the input, so total storage is O(n) when considering the input arrays.)

Note: This code assumes activity times are non-negative because it initializes `lastFinish = -1`.